

IBM Machine Learning Professional Certificate
Supervised Machine Learning: Regression - Course Project

A Project Report

on

**Comparative Analysis of Regression
Models on Housing Prices Dataset**

by

Aryan Agarwal

July, 2025

Table of Contents

1. Introduction and Summary of the Dataset	1
2. Data Preprocessing	3
3. Linear Regression	5
4. Scaled Linear Regression	6
5. K-fold Cross Validation	7
6. ML Pipeline	8
7. Regularization	9
8. Key Findings and Conclusion	11

1. Introduction and Summary of the Dataset

1.1 About the Project

In this project, the aim will be to build the best possible Machine Learning model on a given dataset, by building and testing various different models employing a multitude of methods and processes. The insights from each of the models shall be used to create the model with the best performance.

Every model will be tested by performing the R-squared (R^2) test on them, and the results will be tabulated into a dataframe, to be compared at the very end to find the best model.

1.2 Information about the Dataset

The dataset chosen for this project is the Housing Prices Dataset, available on Kaggle. This dataset contains basic information about houses in an area, as well as the house price. The columns include the area, no. of bedrooms and bathrooms, whether or not facilities such as hot water, air conditioning, basements are available etc.

The column 'Price' will function as the target variable in this project, and the aim will be to predict as closely as possible, the value of this column.

The dataset is available on the following URL:

<https://www.kaggle.com/datasets/yasserh/housing-prices-dataset>

1.3 Additional Information

This project was coded on the web-based Jupyter Notebook IDE, using the Python (Pyodide) kernel.

The coding was done in Python 3.13.3, employing the following libraries:

- Pandas
- NumPy
- Matplotlib
- Seaborn
- SciPy
- Scikit-learn

1.4 Dataset Summary

This dataset spans 545 rows, each representing one house, by 13 columns, representing 12 features and the house price. 6 of the columns are of 'int64' datatype while the remaining 7 are 'object' datatype.

As seen below, there are no missing values present in the dataset.

```
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   price               545 non-null    int64
1   area                545 non-null    int64
2   bedrooms            545 non-null    int64
3   bathrooms           545 non-null    int64
4   stories             545 non-null    int64
5   mainroad            545 non-null    object
6   guestroom           545 non-null    object
7   basement            545 non-null    object
8   hotwaterheating     545 non-null    object
9   airconditioning     545 non-null    object
10  parking             545 non-null    int64
11  prefarea            545 non-null    object
12  furnishingstatus    545 non-null    object
dtypes: int64(6), object(7)
```

The numeric columns have the following data distribution:

	price	area	bedrooms	bathrooms	stories	parking
count	5.450000e+02	545.000000	545.000000	545.000000	545.000000	545.000000
mean	4.766729e+06	5150.541284	2.965138	1.286239	1.805505	0.693578
std	1.870440e+06	2170.141023	0.738064	0.502470	0.867492	0.861586
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	0.000000
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	0.000000
50%	4.340000e+06	4600.000000	3.000000	1.000000	2.000000	0.000000
75%	5.740000e+06	6360.000000	3.000000	2.000000	2.000000	1.000000
max	1.330000e+07	16200.000000	6.000000	4.000000	4.000000	3.000000

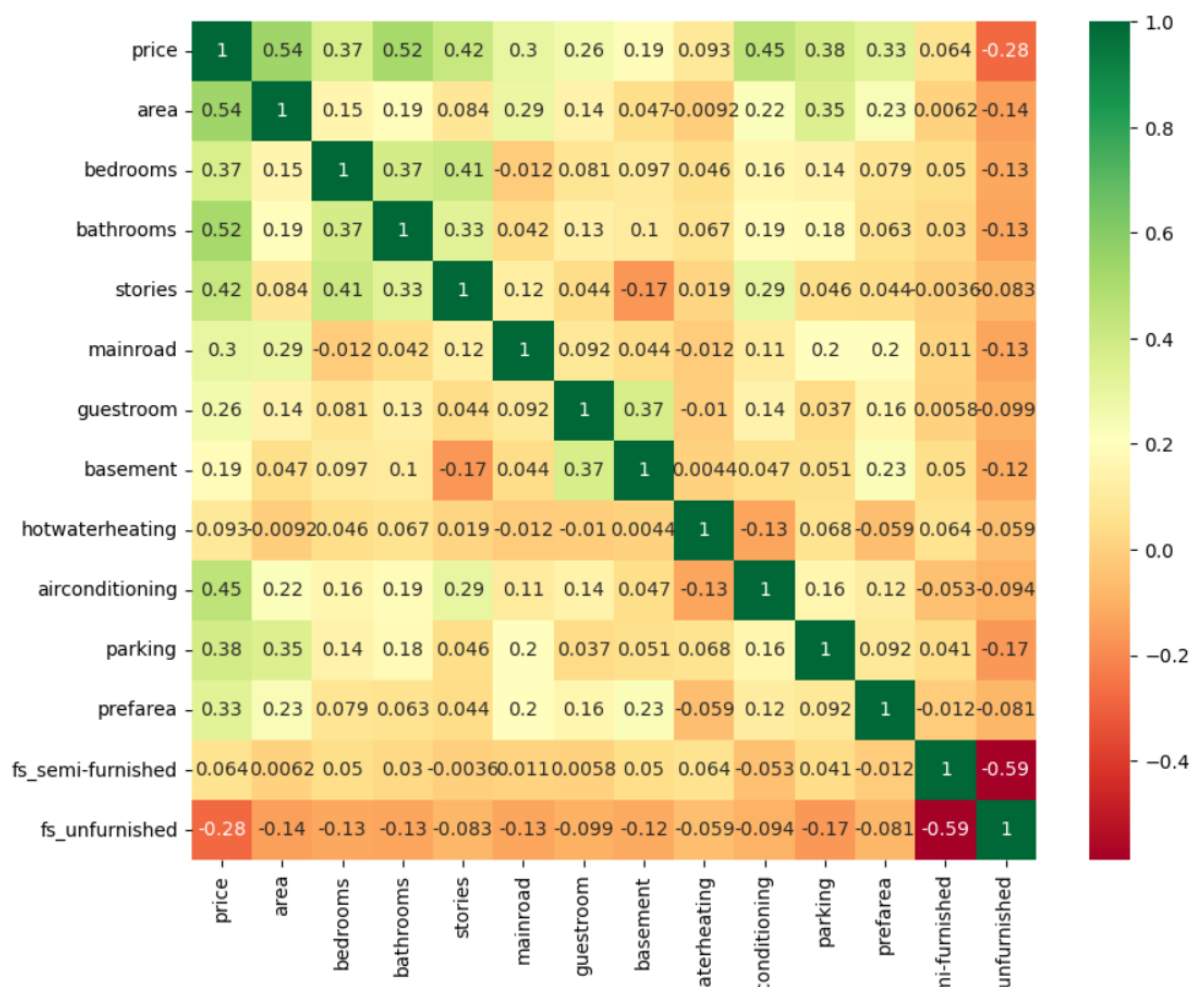
2. Data Preprocessing

The next step will be to preprocess this dataset. Here, two processes were used:

- All the binary 'yes'/'no' columns were mapped to integer values of 1/0 to allow them to be used as features.
- The one categorical column with multiple unique values, 'furnishingstatus', was one-hot encoded using `pd.get_dummies()`.

Now that the dataset was made of only numeric columns, the next step was to split it into features and the target variable and then we quickly looked at the correlations between the different columns, in order to gain a better understanding of which columns are most associated with the target column.

This was achieved using the `.heatmap()` function of Seaborn.



As seen above, the 'area' and 'bathrooms' columns seem to be the most highly correlated with 'price', along with amenities such as 'airconditioning' and 'parking'.

The final preprocessing step was to add some polynomial features to the dataset, in order to capture any non-linear features and relationships between features that may be present. This was achieved using the PolynomialFeatures library of Scikit-learn, with a degree of 2 applied.

3. Linear Regression

With a cleaned, preprocessed dataset with added polynomial features, the first model was then trained. In this section, the models were trained using the 'LinearRegression()' library without scaling or regularizing.

Before moving ahead with the model, a dataframe was created to store two metrics for all the different models we would train ahead, namely the R-squared test score, as well as the root mean squared error (RMSE).

3.1 Model Training

First, the 'train_test_split()' function was used to segment the datasets. Next, the LinearRegression() function was used to create an object of the function. Then the model was fitted, and used to make predictions.

The model was trained and tested on both datasets, with and without polynomial features, in order to test whether the addition of these features was fruitful.

3.2 Model Performance

Finally, the predictions will be subjected to the R-squared test, and the results were tabulated.

	Model	R2_Score	RMSE
0	LR	0.674046	9.753212e+05
1	LR with PF	0.181573	1.545467e+06

As seen above, the model performed much worse on the test set with polynomial features added, hinting at an absence of non-linear relationships between features, or possible overfitting.

4. Scaled Linear Regression

In this section, the features were using the 'StandardScaler()' library. The aim was to figure out what affect, if any, scaling has on the scored metrics for a plain LR model.

	Model	R2_Score	RMSE
0	LR	0.674046	9.753212e+05
1	LR with PF	0.181573	1.545467e+06
2	LR with scaling	0.674046	9.753212e+05

Clearly, scaling had negligible effect on the R2 score for a plain linear regression model. This is true for any method of scaling, as demonstrated when the model was scaled with Standard Scaler, Min-Max Scaler, as well as MaxAbs scaler:

	Model	R2_Score	RMSE
0	LR	0.674046	9.753212e+05
1	LR with PF	0.181573	1.545467e+06
2	LR with scaling	0.674046	9.753212e+05
3	LR with scaled (MM)	0.674046	9.753212e+05
4	LR with scaling (MA)	0.674046	9.753212e+05

5. K-Fold Cross Validation

In this section, the model was tested on various different train-test splits. This process was accomplished using the KFold library, with 4 shuffled splits.

[NOTE: Other cross-validation strategies such as StratifiedKfold and GroupKfold were not used, as there was no requirement to preserve class balance or data leakage between groups.]

	Model	R2_Score	RMSE
0	LR	0.674046	9.753212e+05
1	LR with PF	0.181573	1.545467e+06
2	LR with scaling	0.674046	9.753212e+05
3	LR with scaled (MM)	0.674046	9.753212e+05
4	LR with scaling (MA)	0.674046	9.753212e+05
5	LR with KFold	0.648695	1.097882e+06

As can be seen, while there was slight variation in the performance of the model on different validation sets, the difference was very slight, suggesting that the model can perform reasonably well on any chunk of this dataset. Furthermore, the mean scores were almost the same as without cross validation.

Before moving ahead with regularization, one issue that was observed was the growing cumbersomeness of coding the model-building process, owing to the multiple steps that are being performed every iteration. These include scaling the data, fitting the model, and then getting the predictions for every new model.

All of these, along with any other required steps, were simplified by introducing a pipeline.

6. ML Pipeline

In this section, the 'Pipeline()' library was used to simplify the above code and make it more concise. This was achieved by defining each step as parameters in the pipeline.

Thereafter, the 'cross_val_predict()' function was used to make the actual predictions. As expected, the scoring metrics remained almost unchanged, since no new steps were added.

However, the code did simplify and become much more streamlined and readable, without compromising the efficiency of the model.

	Model	R2_Score	RMSE
0	LR	0.674046	9.753212e+05
1	LR with PF	0.181573	1.545467e+06
2	LR with scaling	0.674046	9.753212e+05
3	LR with scaled (MM)	0.674046	9.753212e+05
4	LR with scaling (MA)	0.674046	9.753212e+05
5	LR with KFold	0.648695	1.097882e+06
6	LR using Pipeline	0.652184	1.102097e+06

7. Regularization

The final step in refining the model was to add regularization, to prevent overfitting.

Several models were tested, mainly LASSO and Ridge. These methods allowed the model to perform hyperparameter tuning, in order to get the best parameters.

7.1 Model Training

In order to use the required regularization method, the code remained largely unchanged, and the only change required was the replacement of the linear regression object with the object of the regularization library.

To find the best parameters, the entire pipeline was iterated with multiple alpha values, defined beforehand.

7.2 Model Performance

	Model	R2_Score	RMSE
0	LR	0.674046	9.753212e+05
1	LR with PF	0.181573	1.545467e+06
2	LR with scaling	0.674046	9.753212e+05
3	LR with scaled (MM)	0.674046	9.753212e+05
4	LR with scaling (MA)	0.674046	9.753212e+05
5	LR with KFold	0.648695	1.097882e+06
6	LR using Pipeline	0.652184	1.102097e+06
7	Lasso Pipeline	0.652435	1.859046e+06
8	Ridge Pipeline	0.653059	1.761770e+06

Both Lasso and Ridge regressions proved to give negligible improvement in the model. This could indicate that the model is not overfitted or underfitted on this dataset, and had already achieved close to its best performance.

Finally, all the steps performed throughout this modeling process will be combined, using the powerful 'GridSearchCV()' function of the Scikit_learn library.

Using this, the model shall be iteratively trained and tested on various combinations of pipelines, and alphas, to find the best possible model that can be built on this dataset.

```
Best model : Pipeline(steps=[('Scaler', StandardScaler()),
                             ('Reg', Ridge(alpha=np.float64(100.0), max_iter=10000))])
Best parameters : {'Reg': Ridge(max_iter=10000), 'Reg__alpha': np.float64(100.0)}
R-squared score : 0.670090189504127
```

This completes the model-building process, with an achieved optimum R-squared test score of 0.6701, using a pipeline of standard scaler and ridge regression. However, this was outperformed by the simplest Logistic Regression model, hinting at the fact that this dataset did not contain too many patterns that were discovered by more complex models alone, and simple models were able to perform better.

8. Key Findings and Conclusion

8.1 Insights and Takeaways

In the process of building the optimum model on this dataset, various key insights were revealed, that helped gauge the relationships of features among themselves, as well as with the target column. Following are some of the main takeaways:

- The features of 'area', 'bathrooms', 'airconditioning' and 'parking' were the most highly correlated with the 'price' column.
- On average, the features had mostly a linear relationship with the target variable, with the models performing better on data without the polynomial features added.
- Scaling unregularized data had no effect on its performance, regardless of the method of scaling.
- Lasso and Ridge regularization methods gave slight improvements on the model performance, but not much, indicating that the model was not overfitting.
- The GridSearchCV function gave the optimum model for this dataset as the Ridge regression model with $\alpha = 100$.
- In the end, however, the simpler models proved to be more effective than the more complex ones for this dataset, showing that this dataset did not contain too many patterns that were discovered by more complex models alone, and simple models were able to perform better.

8.2 Summary

This project progressively built and refined ML models on the given dataset, using various different steps and methods, in order to obtain the best possible one, with the highest performance. These were graded using the R-squared test, which were calculated and tabulated in a separate dataframe.

To recap the entire process, the Housing Prices Dataset was chosen as the dataset on which the models would be trained. Then, all the libraries and functions that would be required in the project were imported.

First, the dataset was cleaned and preprocessed, including the encoding of columns and addition of polynomial features. Then, a model was created using simple linear regression. Next, the data was scaled using multiple scalers, to check the effect on its R2 score. Thereafter, the steps were made

concise by using a Pipeline. Finally the model was tested using regularization methods such as Lasso and Ridge, to check for overfitting, before using the GridSearchCV function to find the optimum model with the best performance.

8.3 Next Steps

Now that the modeling is complete, the possible next steps include:

- Using more advanced techniques to improve model performance, such as using ensemble methods, or running automated ML models.
- Building up the Pipeline, and adding functionality to include more features or methods that may appear in the real-world.
- Performing deeper analysis of the model, including residual analysis, analysing the final coefficients and their implication, error slicing to find any particularly troublesome subsets of bad or faulty data.

These steps will ensure that the takeaways from this modeling project are optimally used, and prove beneficial in practical applications further down the line.